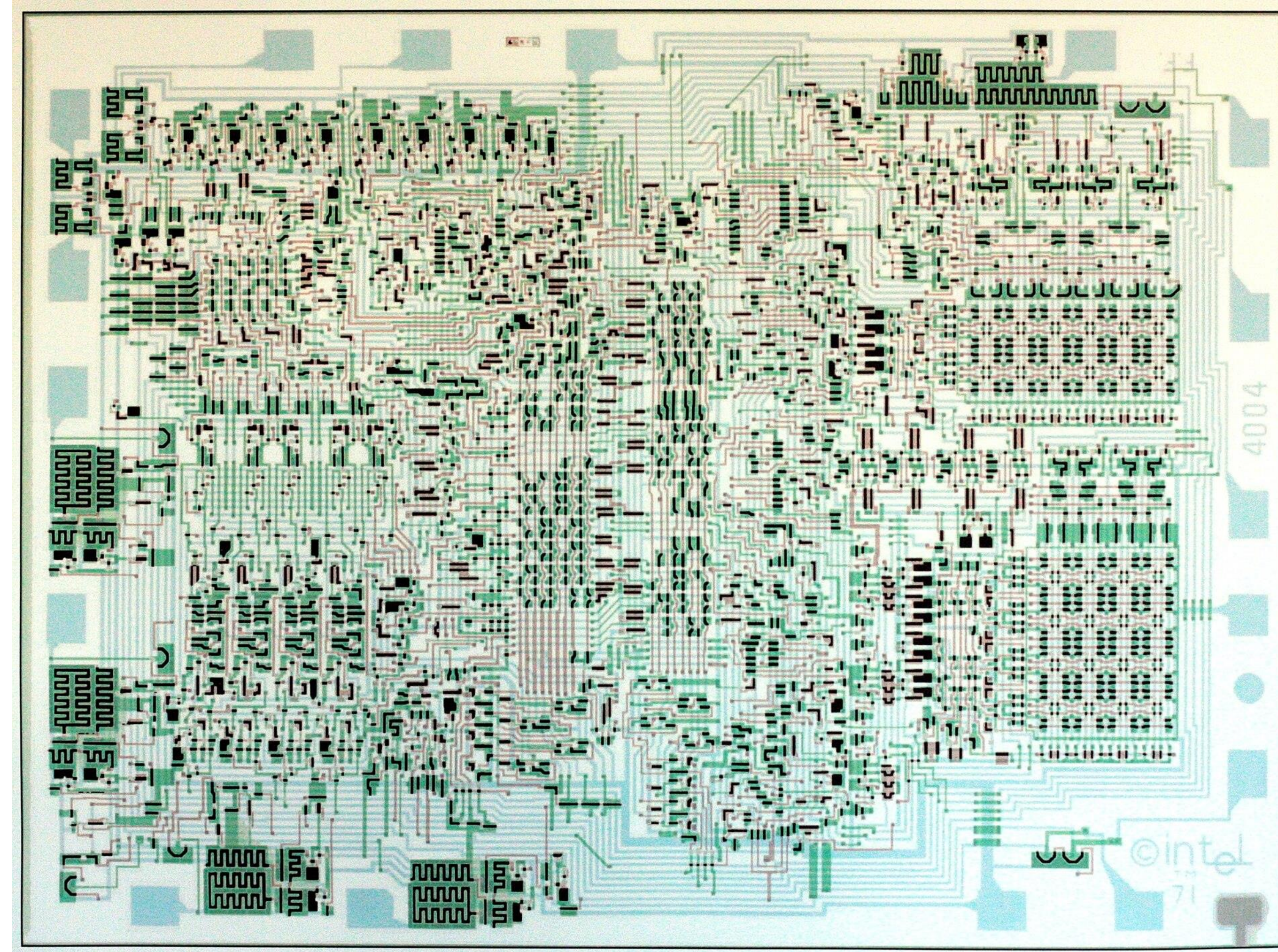


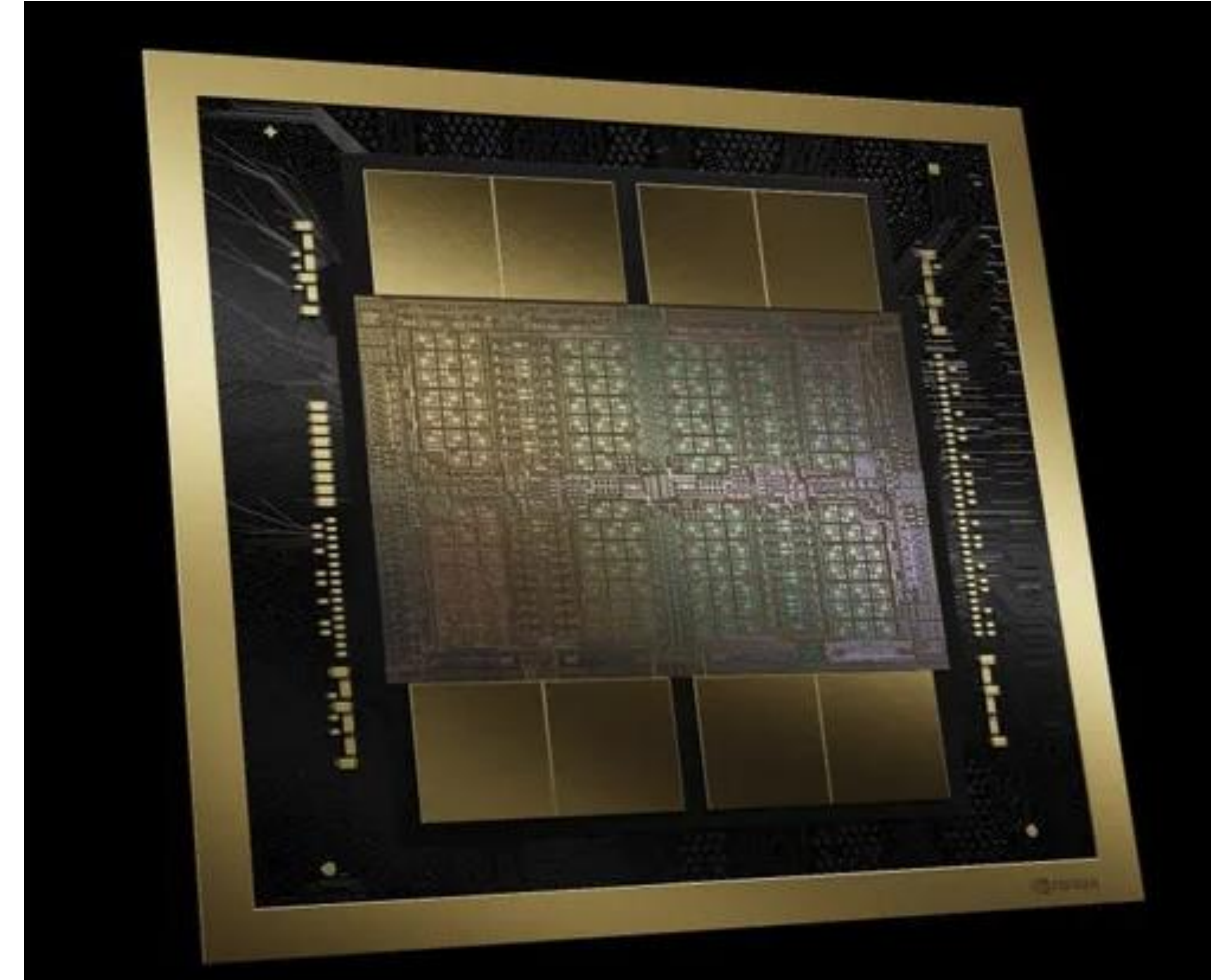
Agentic AI for Chip Design

Agentic Design Automation (ADA) toward Autonomous Design

Chip Design: Million X Productivity Improvement in Half Century



Intel 4004 (1971)
2300 transistors
100K 4B instruction/sec



NVIDIA B200 (2024)
208 Billion Transistors
20 Peta FP4/sec

EDA Driving Force: Increasing Automation and Scaling

- 1973: SPICE, **automate** circuit simulation
- 1986: Synopsys Design Compiler, **automate** logic synthesis
- 1989: Cadence Verilog-XL, **automate** RTL simulation
- 1997: Cadence Silicon Ensemble , **automate** placement & route
- 1998: Synopsys PrimeTime , **automate** static timing analysis
- 2004: Apache RedHawk, **automate** power grid analysis, scaling with multi-threading
- 2008: Synopsys PrimeTime , **scaling** STA with multi-threading
- 2012: Cadence Innovus, **scaling** P&R with extensive multi-threading
- 2016: Ansys RedHawk-SC, **scaling** power grid analysis with distributed many cores

AI/LLM Enables More Automation

Generating insights, knowledge, and ideas. Answer questions about designs, infrastructures, tools, flows, HW domains, etc.

Understanding

Generating design and verification code (RTL, SVA, testbenches, EDA scripts, tools scripts, and configs)

Coding

Summarization, report/log analysis, visualization of design and related data, check rule violations, etc.

Analysis

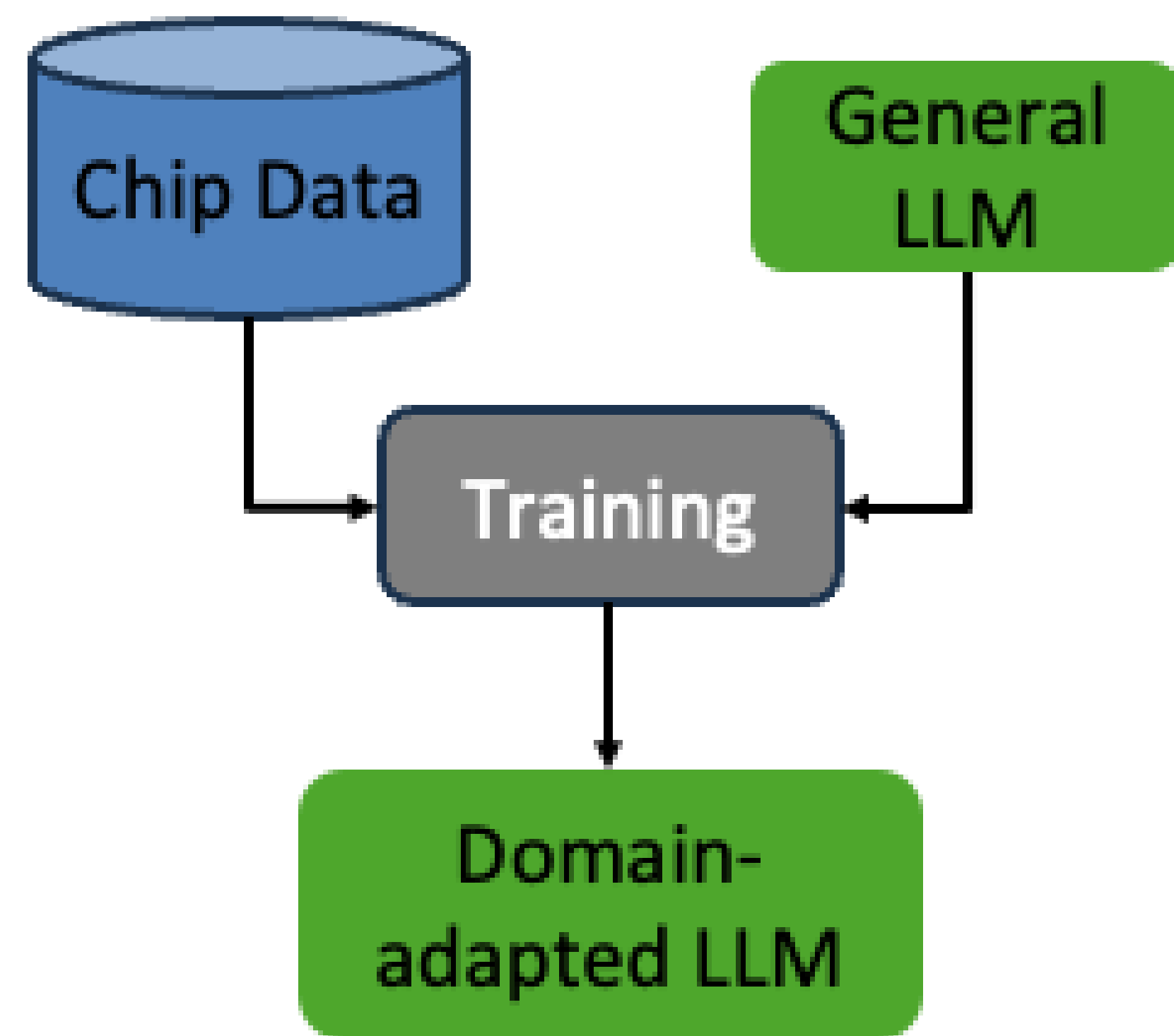
Triage and fix a design problem, e.g. regression failures for logic design and timing failures for physical design.

Debug

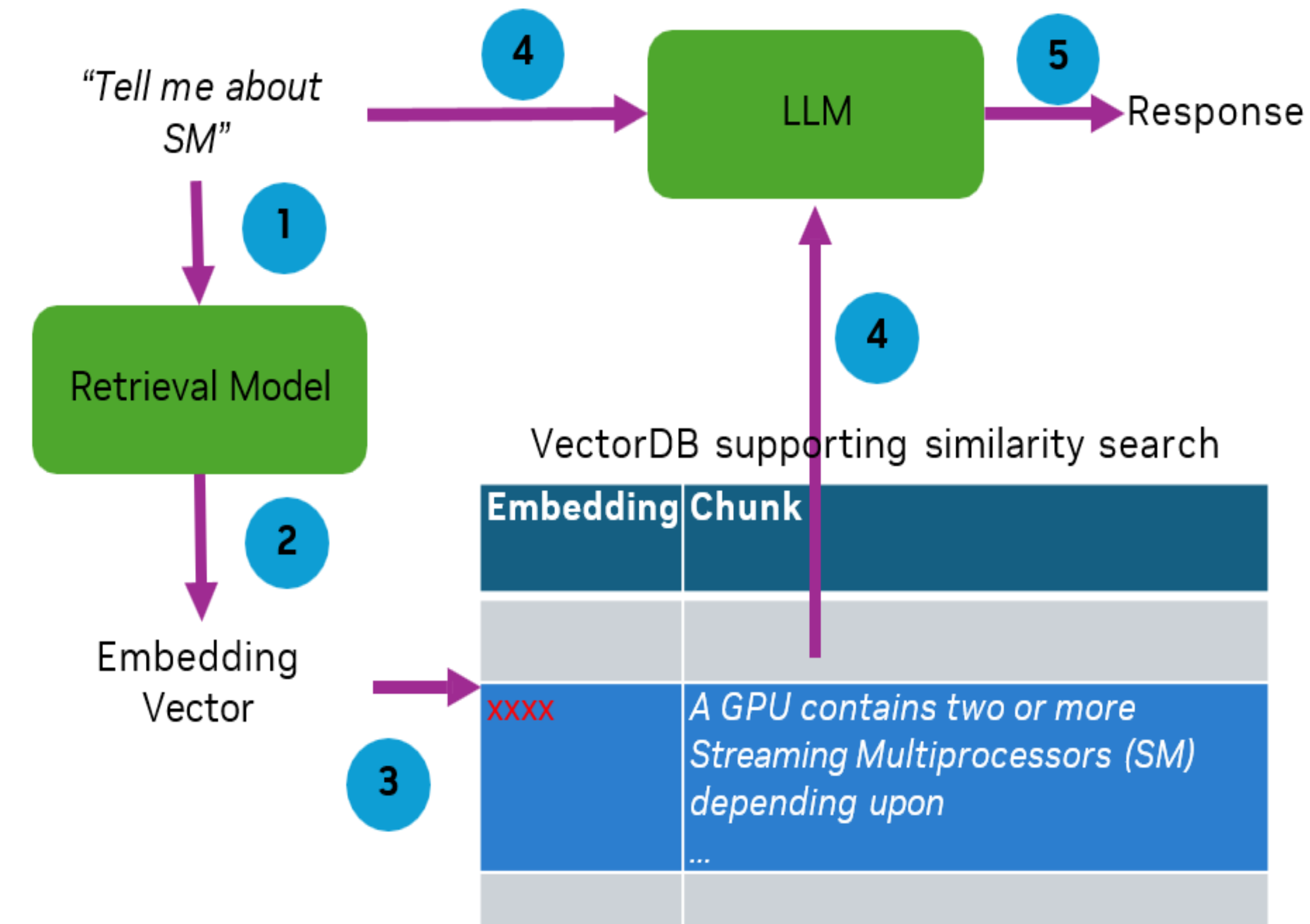
Optimize for design and verification: coverage closure, recipe optimization, architecture exploration, etc.

Optimization

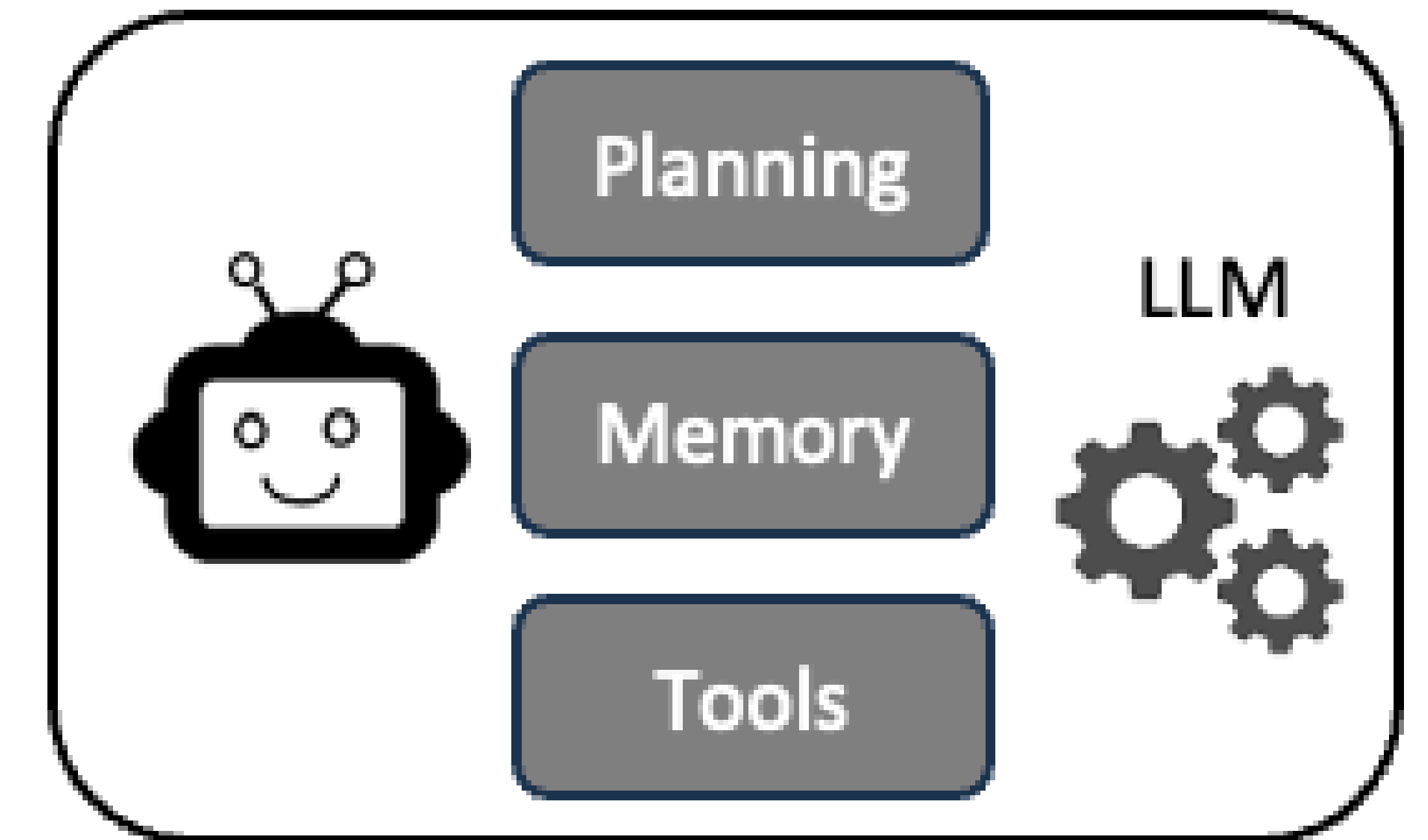
The Progression of LLM Adoption in Chip Design



Parameter
Training
(ChipNemo)



Prompt engineering, In-
Context Learning, RAG
(Chatbots)



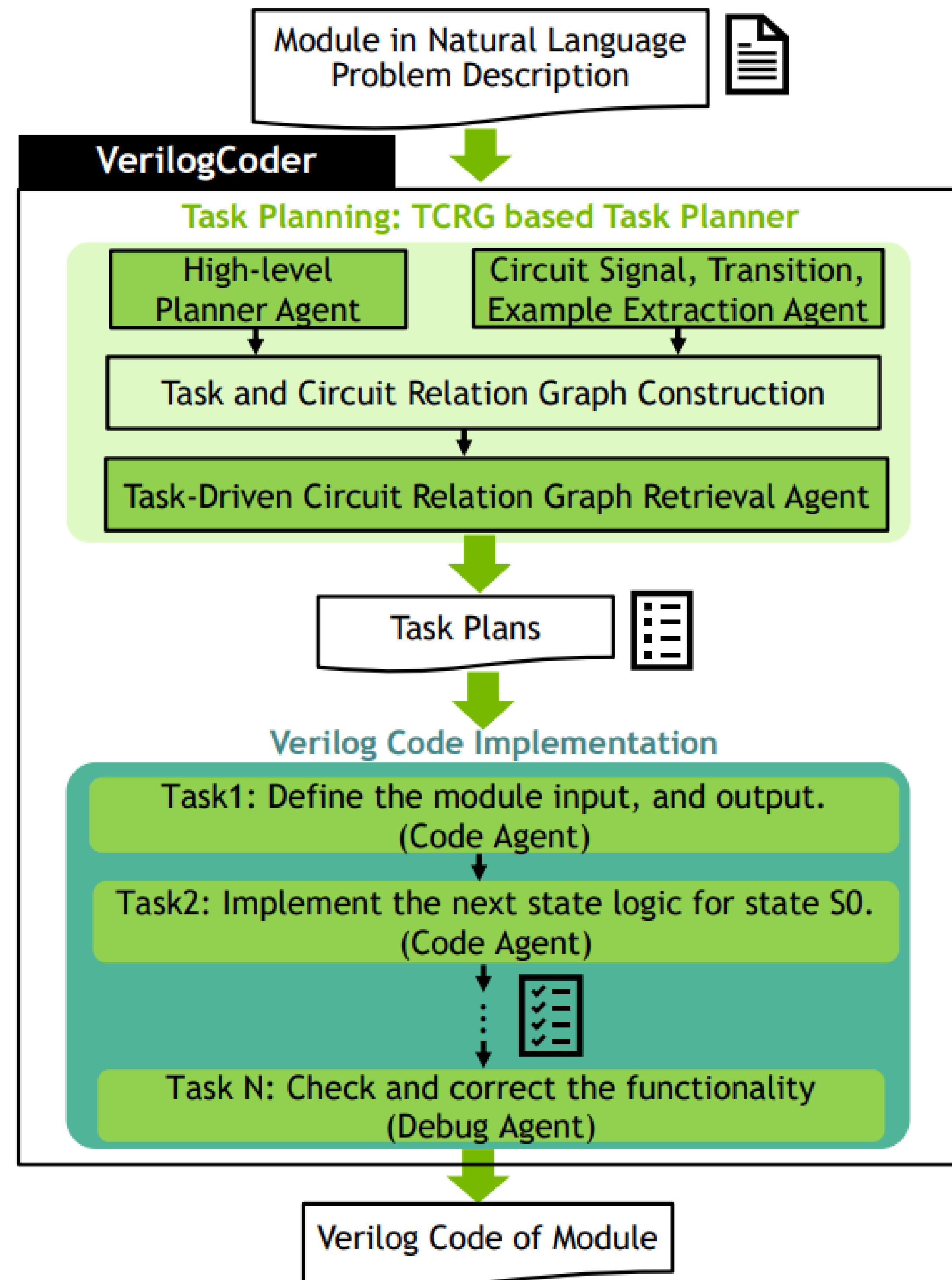
Agent
(Claude Code)

2022

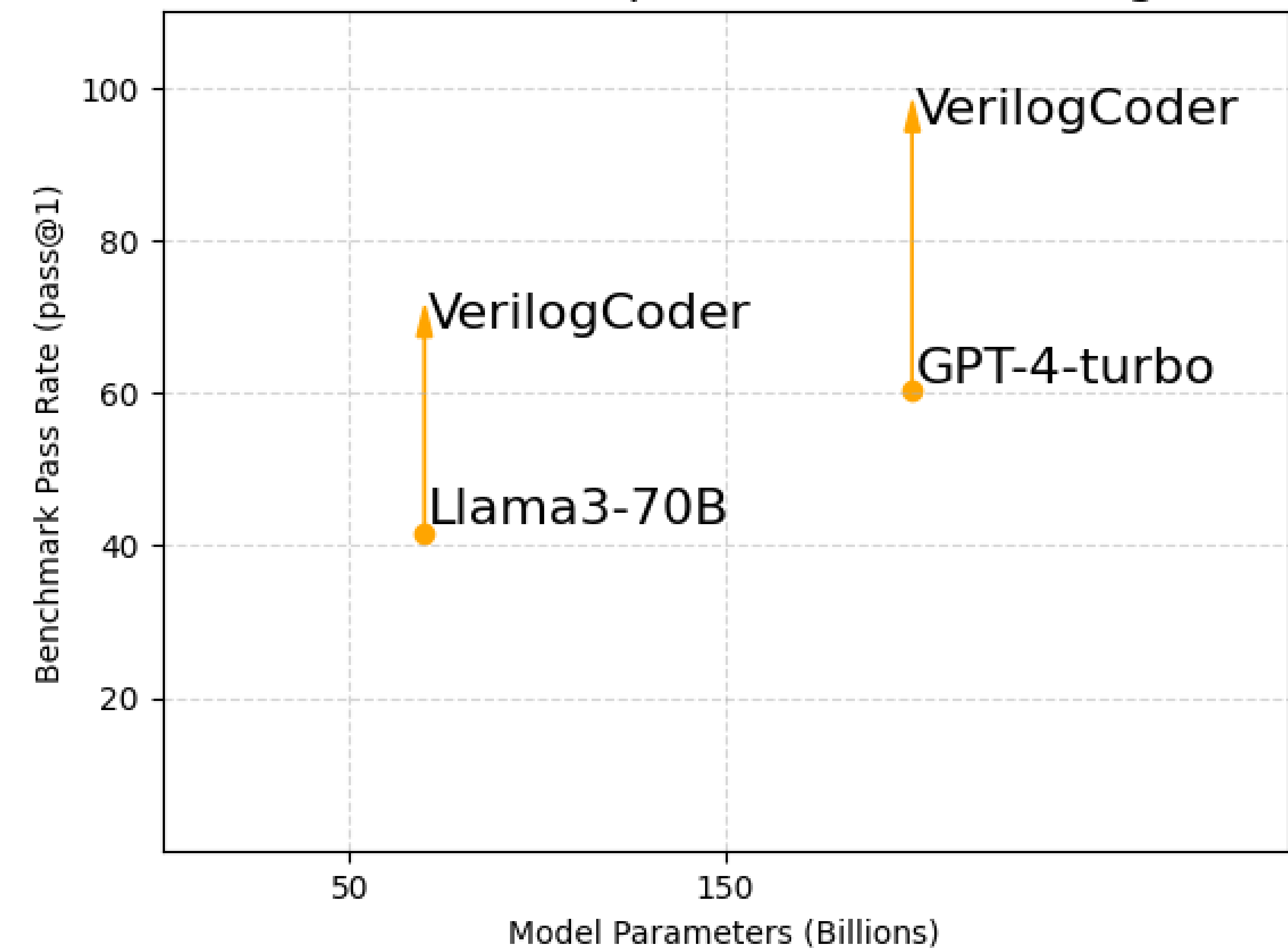
2026

Agent is the Game Changer

VerilogCoder: Agent solved most of VerilogEval problems

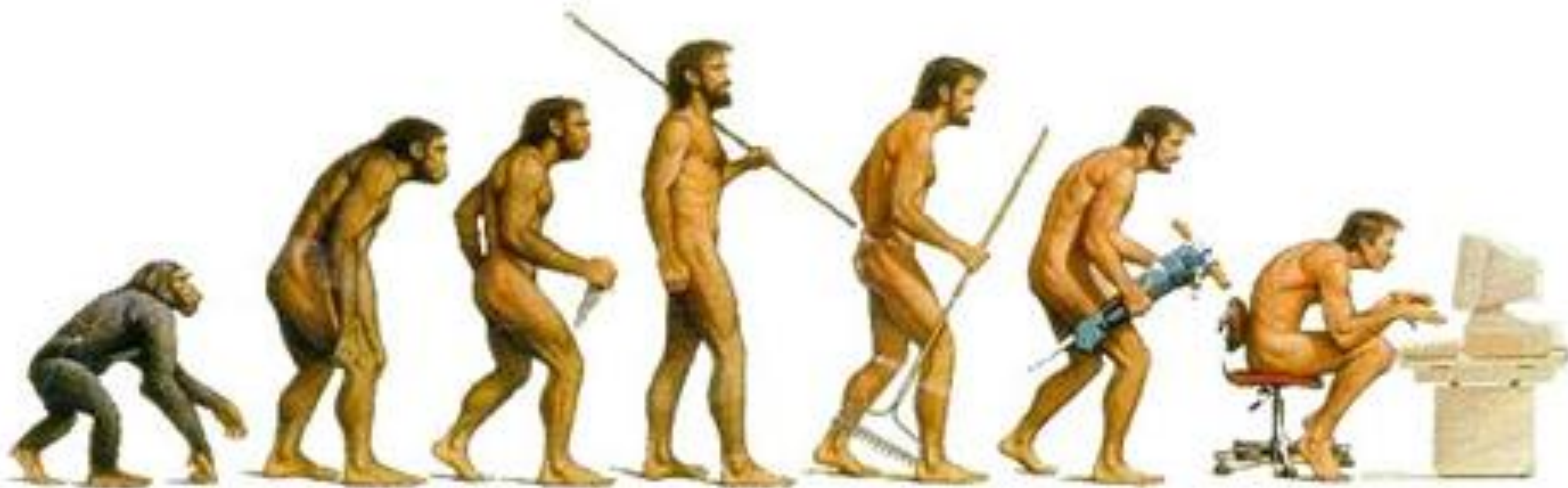


Specification-to-RTL Pass Rate
Arrows indicate improvement of VerilogCoder



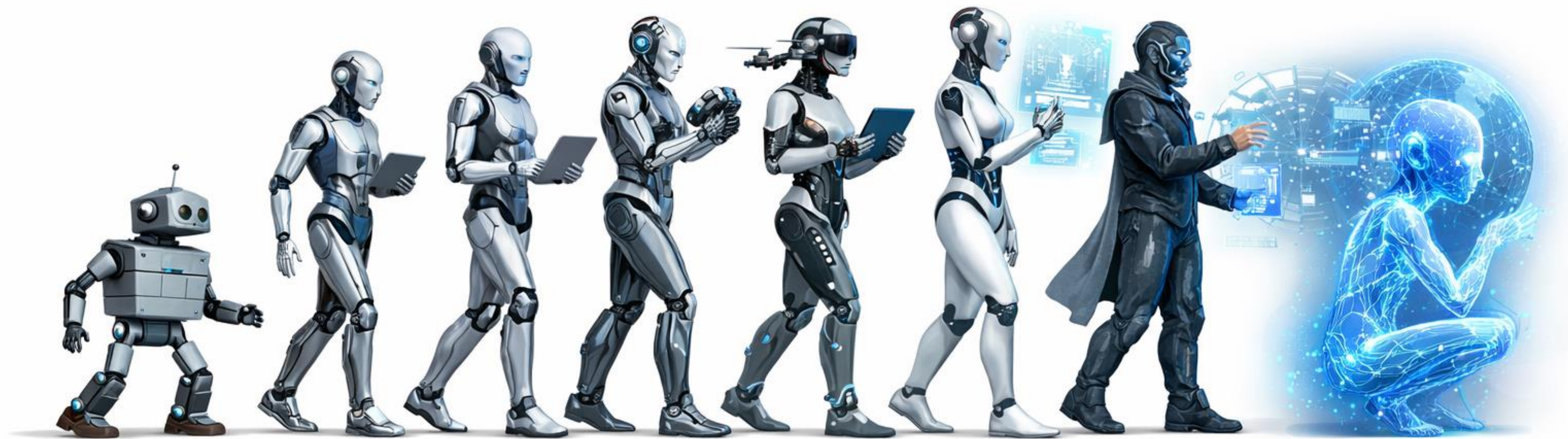
Human Evolution with Tools

Power of human comes from tools we use



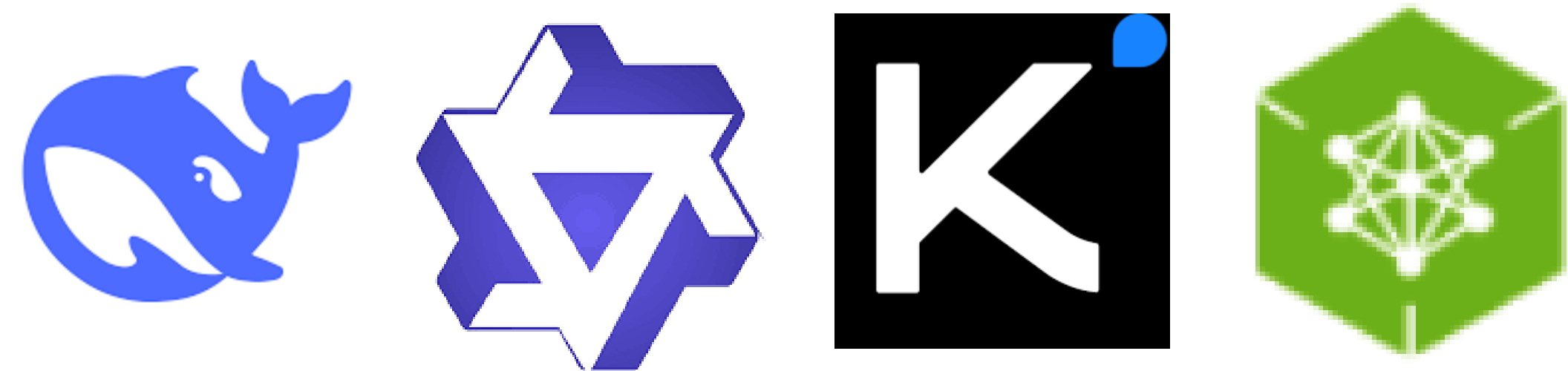
Agent Evolution with Tools

Power of Agents will come from tools they use



Agentic Design Automation (ADA)

Scale the Power of AI and Agent with Custom Models and Agent Native Tools



Custom Models

Specialized
Low cost



Frontier Models

General purpose
Expensive

Agents



Agent-Native Tools

Lightweight
Versatile



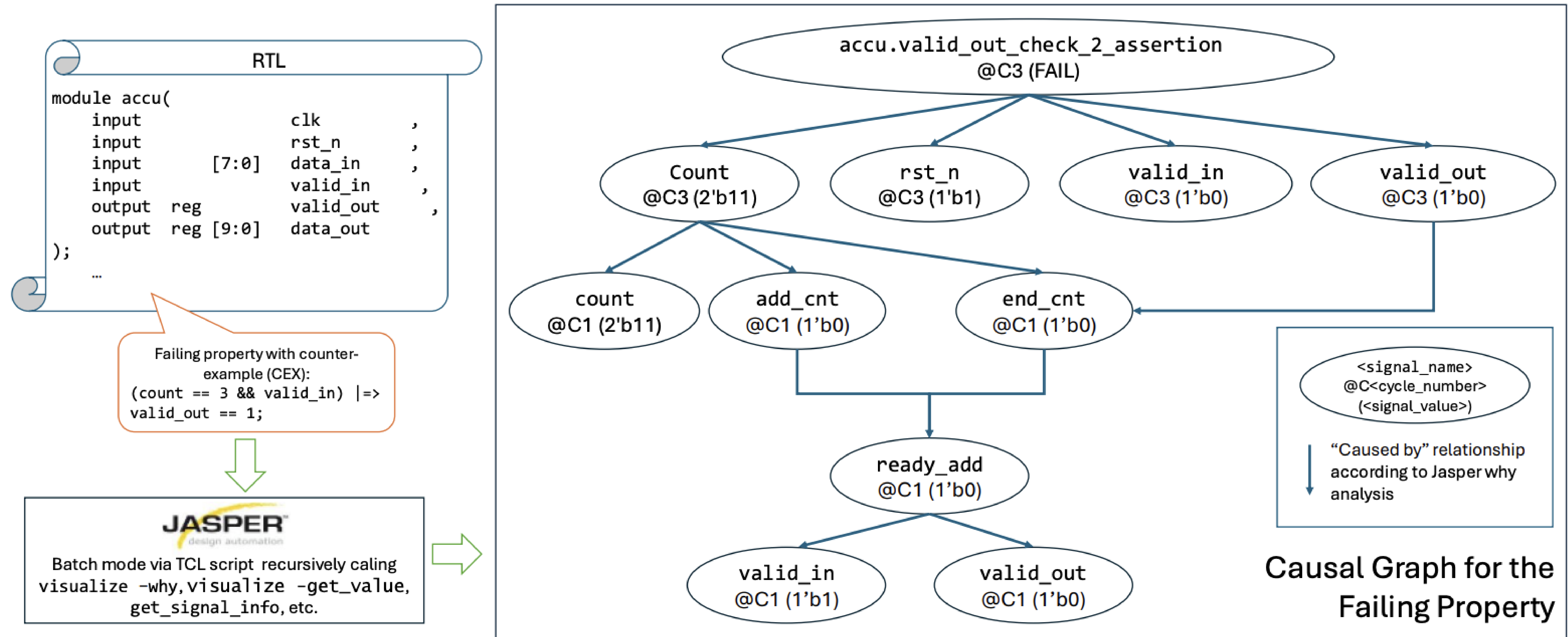
EDA Tools

Heavy-duty
Specialized

Verification Debug

Tools provide context information for debugging

Debug with **causal graph tools** working with EDA tools



Verification Debug

Agent reasons with the causal graph

Agent traverse graph to locate RTL and reasoning about the bug; then propose debug hypothesis

Case Study on accumulator

```
## Buggy Line
assign ready_add = valid_out | !valid_in;
## Fixed Line
assign ready_add = !valid_out | valid_in;
```

Golden Answer

Ranked Hypotheses

Hypothesis 1: Ready_add Logic Issue Preventing valid_out Assertion

****Confidence:** 48.0%**

1 Hypothesis

The valid_out signal remains low because the ready_add condition (ready_add = valid_out OR !valid_in) does not become true when count reaches 3, causing end_cnt to never trigger a valid_out high; this indicates an RTL bug where the handshake logic for ready_add is flawed.

2 Evidence

Supporting Evidence:

- The RTL code 'assign end_cnt = ready_add && (count == 'd3)' implies that even if count reaches 3, end_cnt depends on ready_add being true. This supports the hypothesis if ready_add remains low, preventing valid_out from becoming high.

...

Causal Chain Timeline

...

3. Cycle 1

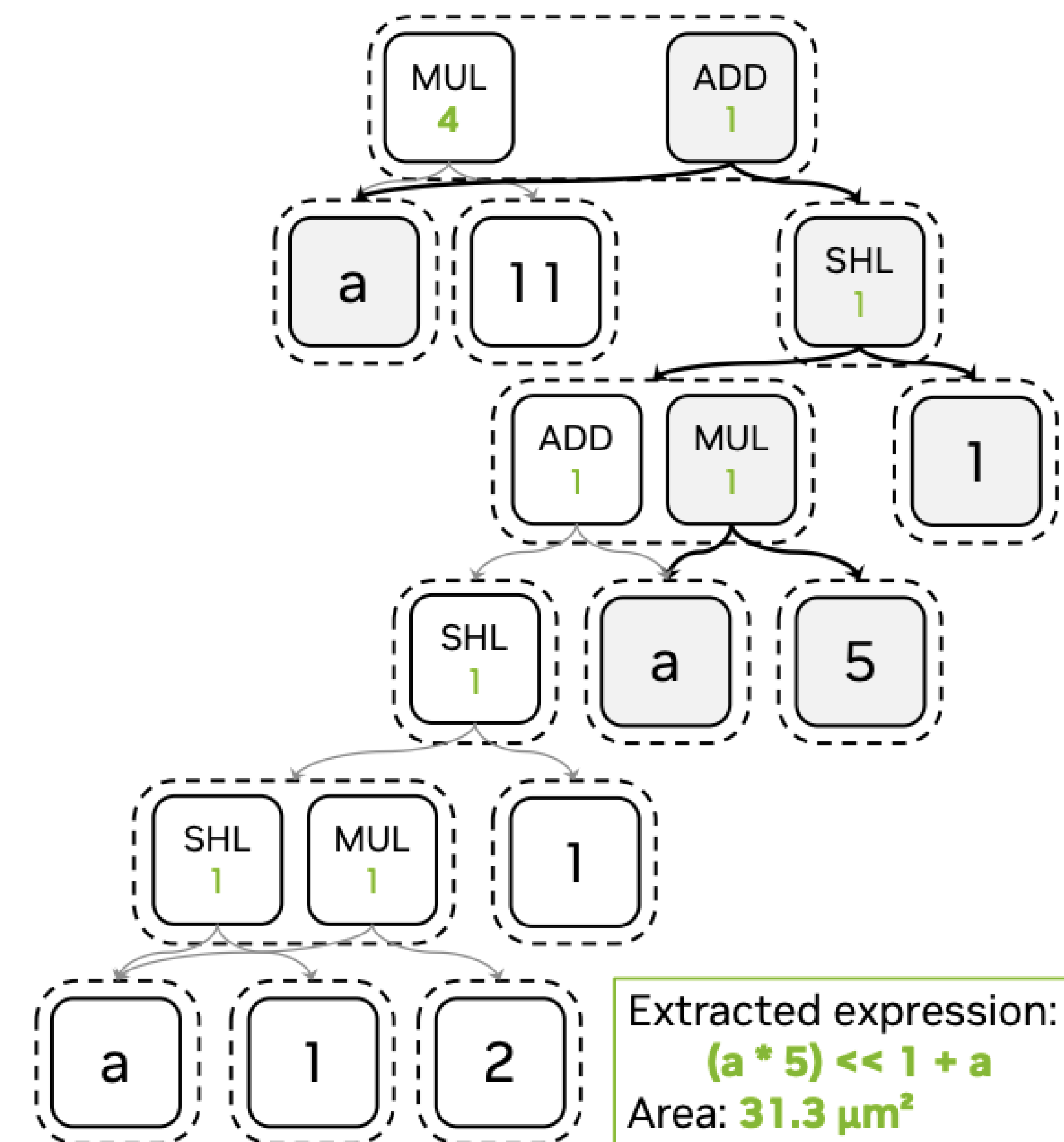
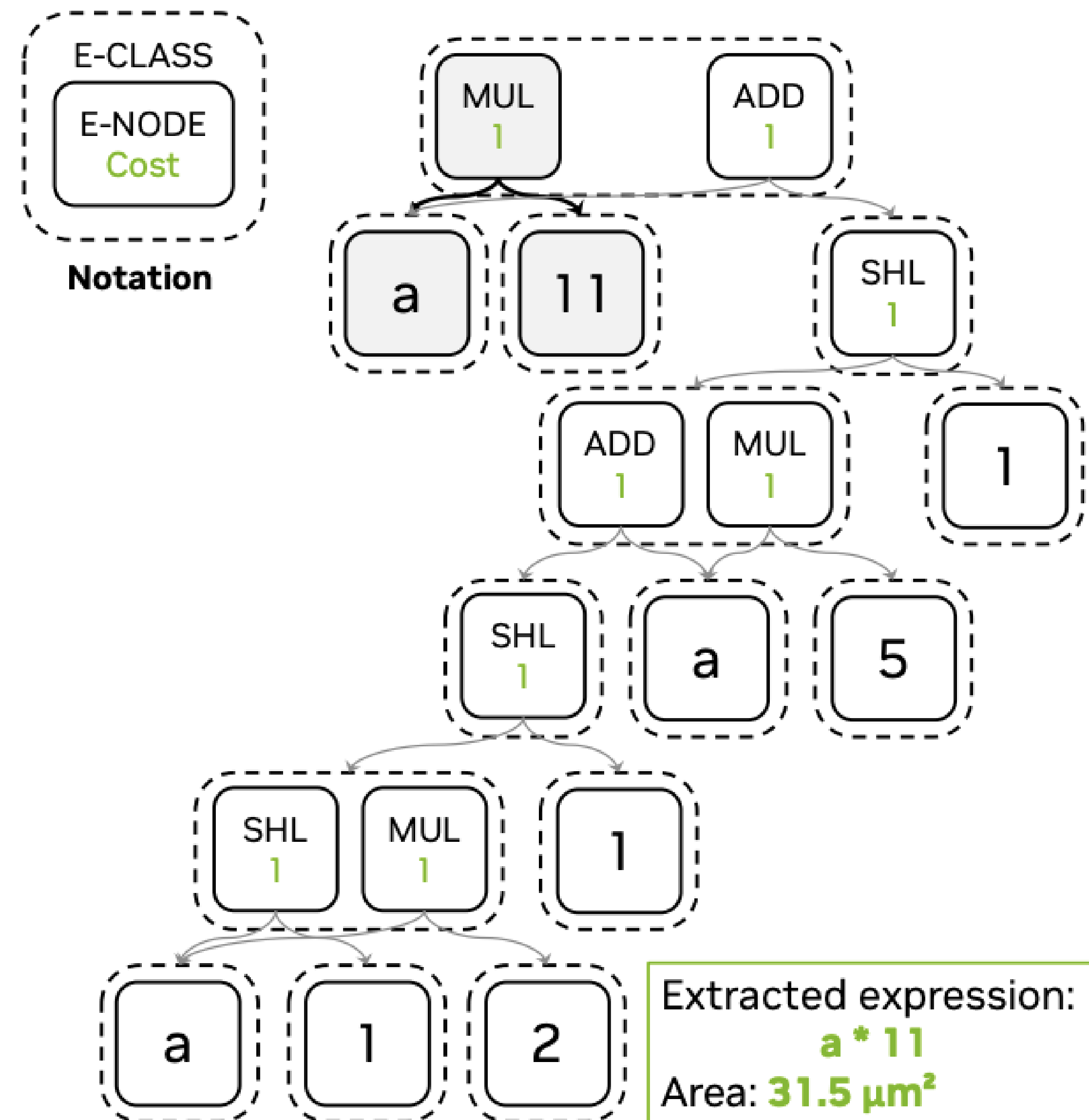
At cycle 1, with valid_in high, the node 'ready_add' evaluates to 1'b0 because it is computed as valid_out OR !valid_in. Since valid_in is high, !valid_in is false and valid_out is low, which blocks accumulation and consequently prevents the triggering of end_cnt and the high assertion of valid_out.

FVDebug Output

Tools conducts optimization

Leverage **e-graph** for correct-by-construction RTL rewriting with E-Graph

E-graph rewriting tools executes the RTL optimization.



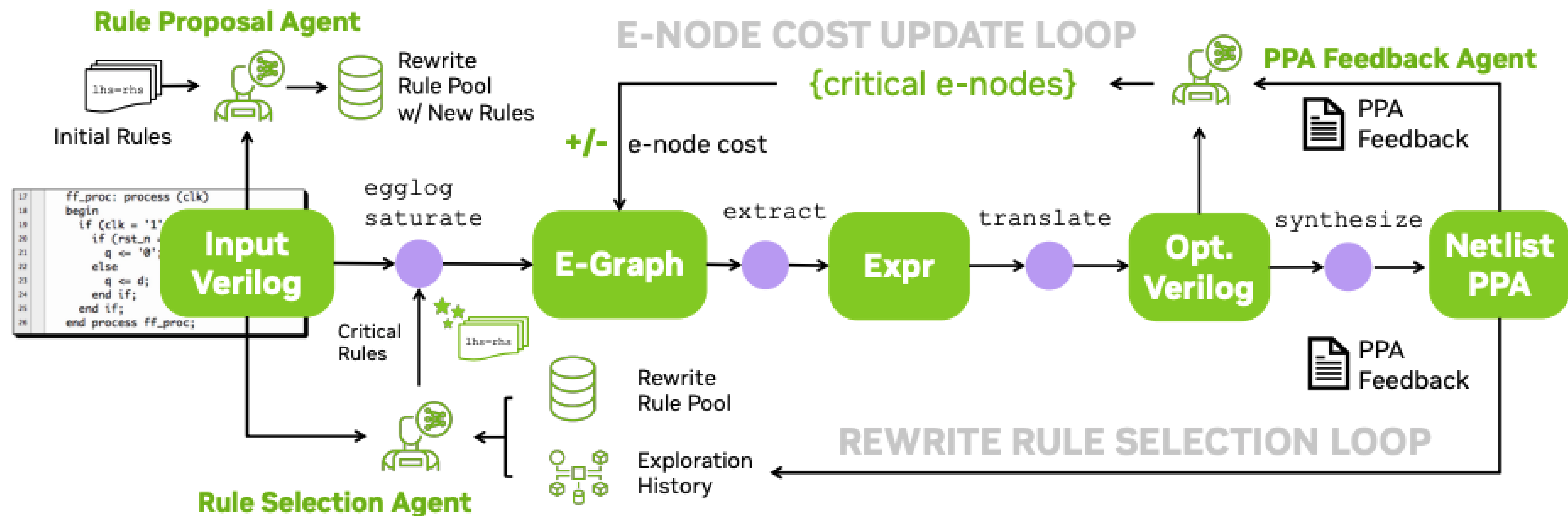
RTL Optimization

Agent drives the e-graph rewrite tools

LLM assisted PPA feedback

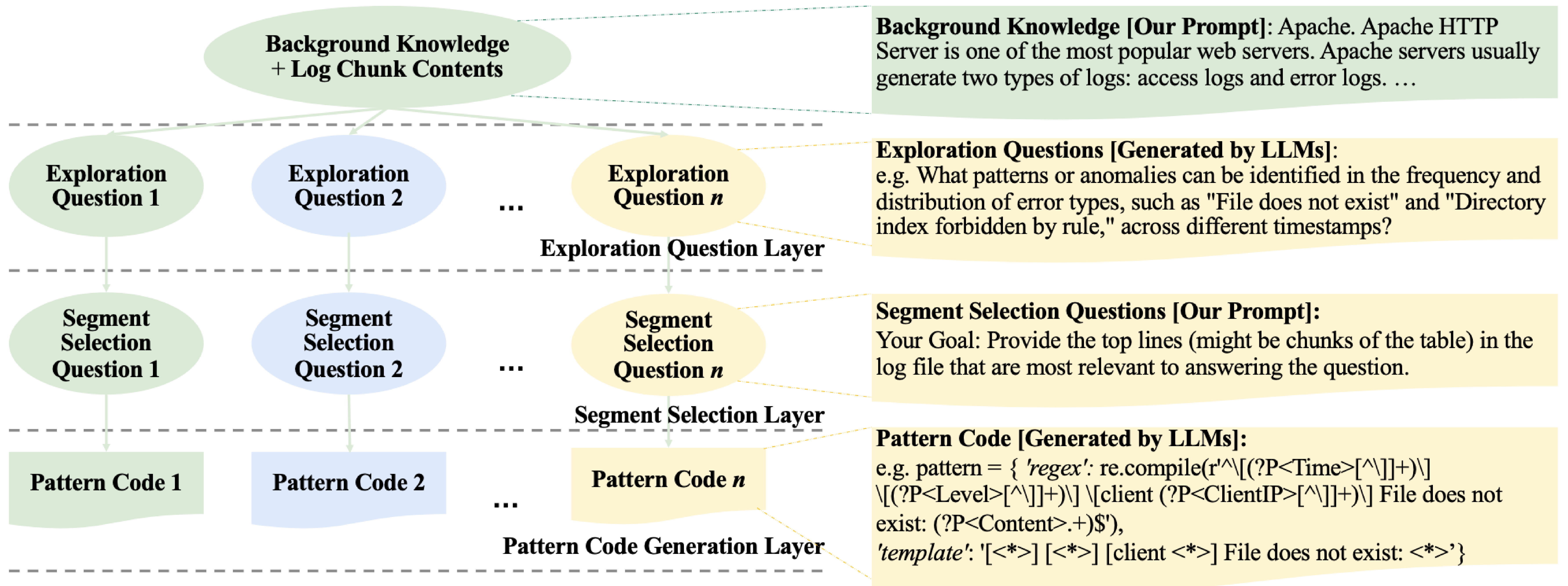
LLM assisted rewrite rule selection and generation

Results: -24.23% area / -12.43% delay vs e-graph only baseline



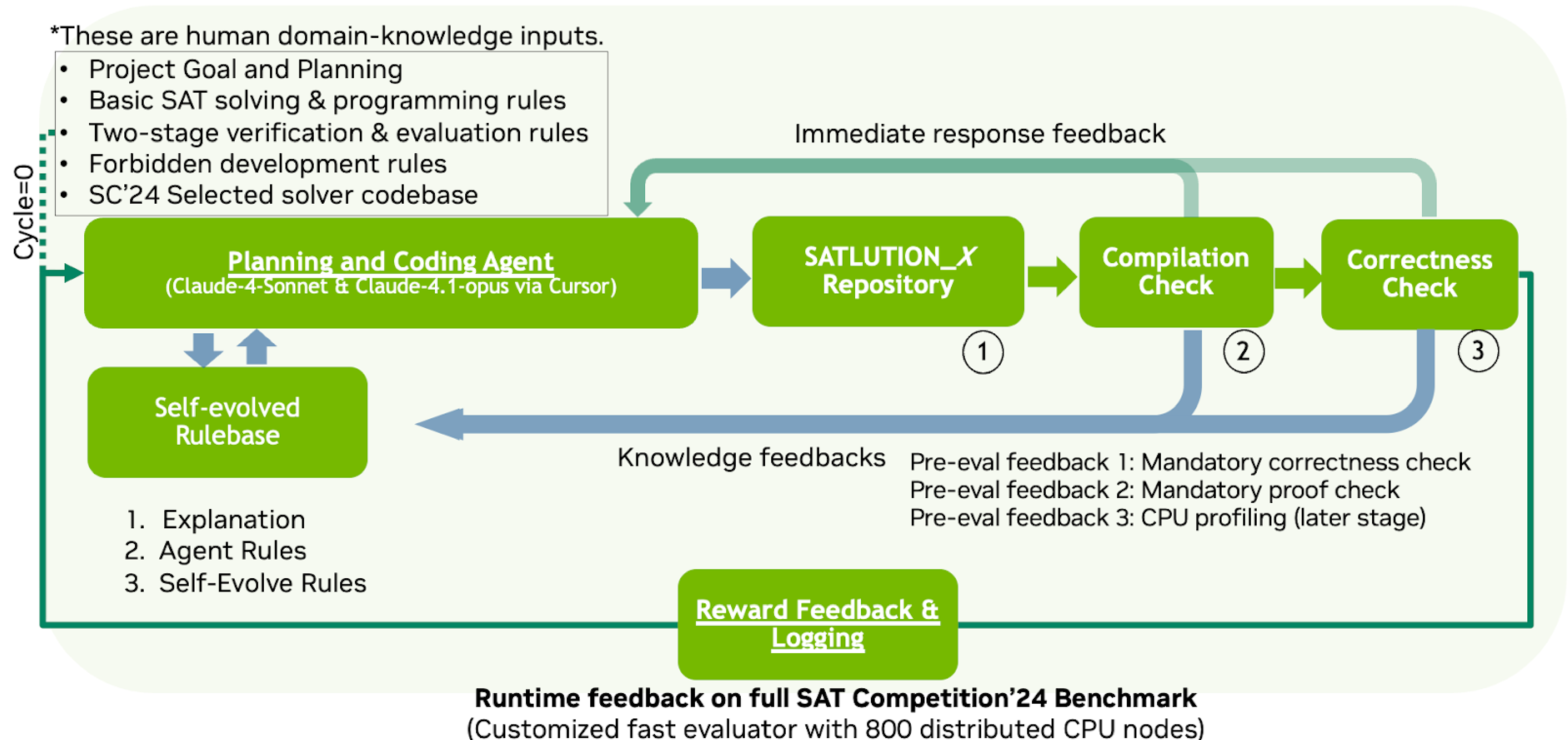
Log Analysis

Agent generates tools/code to extract structure information from log files



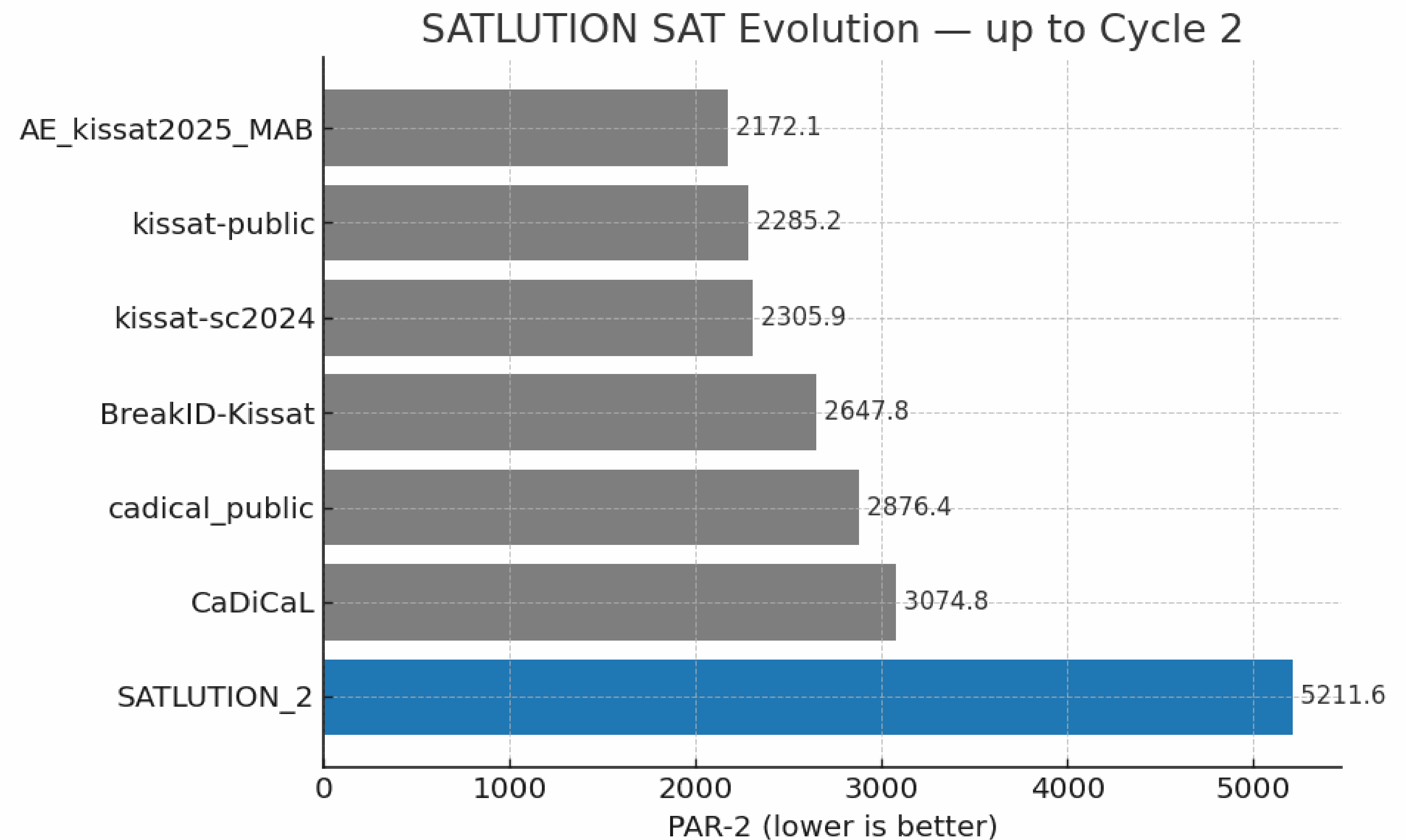
Improve SAT Solver

Agent improves existing tools



SATLUTION : 10+% better than 2025 SAT Competition winners

Start from the winning solvers
in 2024 SAT competition
Improved performance by
18% in a month of evolution



ScaleRTL: Scaling LLMs with Reasoning and Test Time Compute

Train a custom model

Phase 1: Fine-Tuning on Large-Scale CoT Data

Training Sample

Training Sample

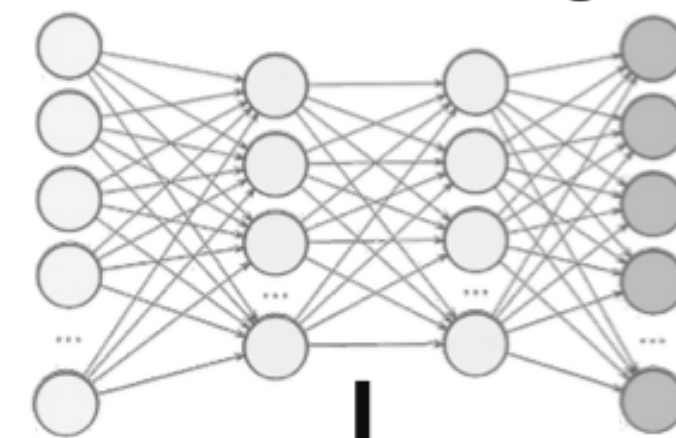
Training Sample

Specification
Build a counter that counts from 0 to 999, inclusive, with a period of 1000 cycles. The reset input is active high synchronous, ...

CoT Reasoning Trace
<think>
Okay, so I need to design a counter that goes from 0 to 999 ...
* First, the counter has to increment on every clock cycle. So ...
* That makes sense. But how are the bits handled? Since the output ...
* Wait, when you write 999, ...
* Let me think about edge cases. What happens when q is 999 ...
* Testing scenarios: reset is high, then q stays 0. When reset is low, ...
Yes. The code should handle all that.
</think>

Code Solution
<answer>
always @(posedge clk) begin
 if (reset) begin
 q <= 10'b0;
 end else begin
 if (q == 999) begin
 q <= 0;
 end else begin
 q <= q + 1;
 end
 end
end
</answer>

General Reasoning Model

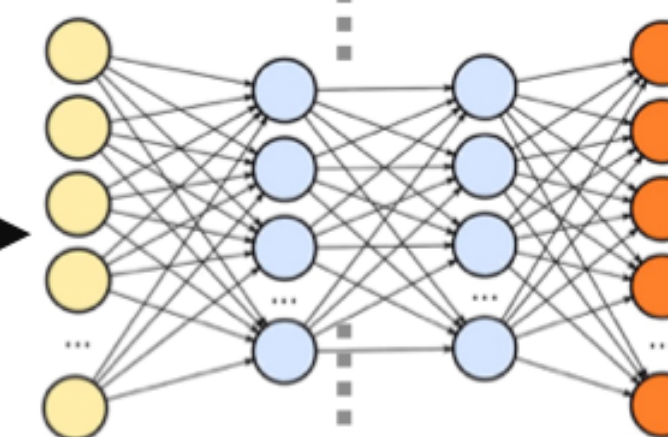


ScaleRTL
(capable of RTL reasoning
via CoT fine-tuning)

Phase 2: RTL Reasoning with Test-Time Scaling

Test Sample

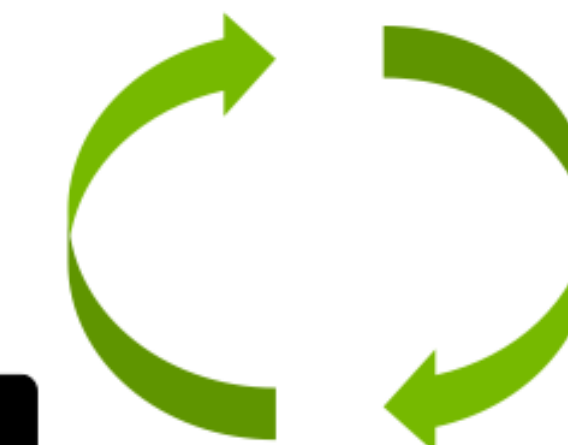
Specification
Create a set of counters suitable for use as a 12-hour clock
(with am/pm indicator) ... hh, mm, and ss are two BCD (Binary-
Coded Decimal) digits each for hours (01-12), minutes (00-59),
and seconds (00-59). Reset is the active high synchronous ...



ScaleRTL[†]
(Enhanced RTL reasoning
with test-time scaling)

Corrective Prompt for Reasoning Extension

<think>
Alright, let's tackle this problem. The goal is ..., let's break it down step by step
* First, I need to model the seconds, minutes, and hours counters, ...
* Starting with the reset condition: when reset is high, the time should be ...
* Wait, but how to handle the BCD increment? Let's think ...
* But this could get complex. Alternatively, handle the entire increment as ...
* Wait, but how to handle BCD increment for hours. For example, ...
* ...
Yes. This covers the necessary transitions. So the code should work.
Wait, my reasoning must be incorrect. Let's check the reasoning rules: {Reasoning Rules}. Wait, I did not follow the rules. Let's try again.

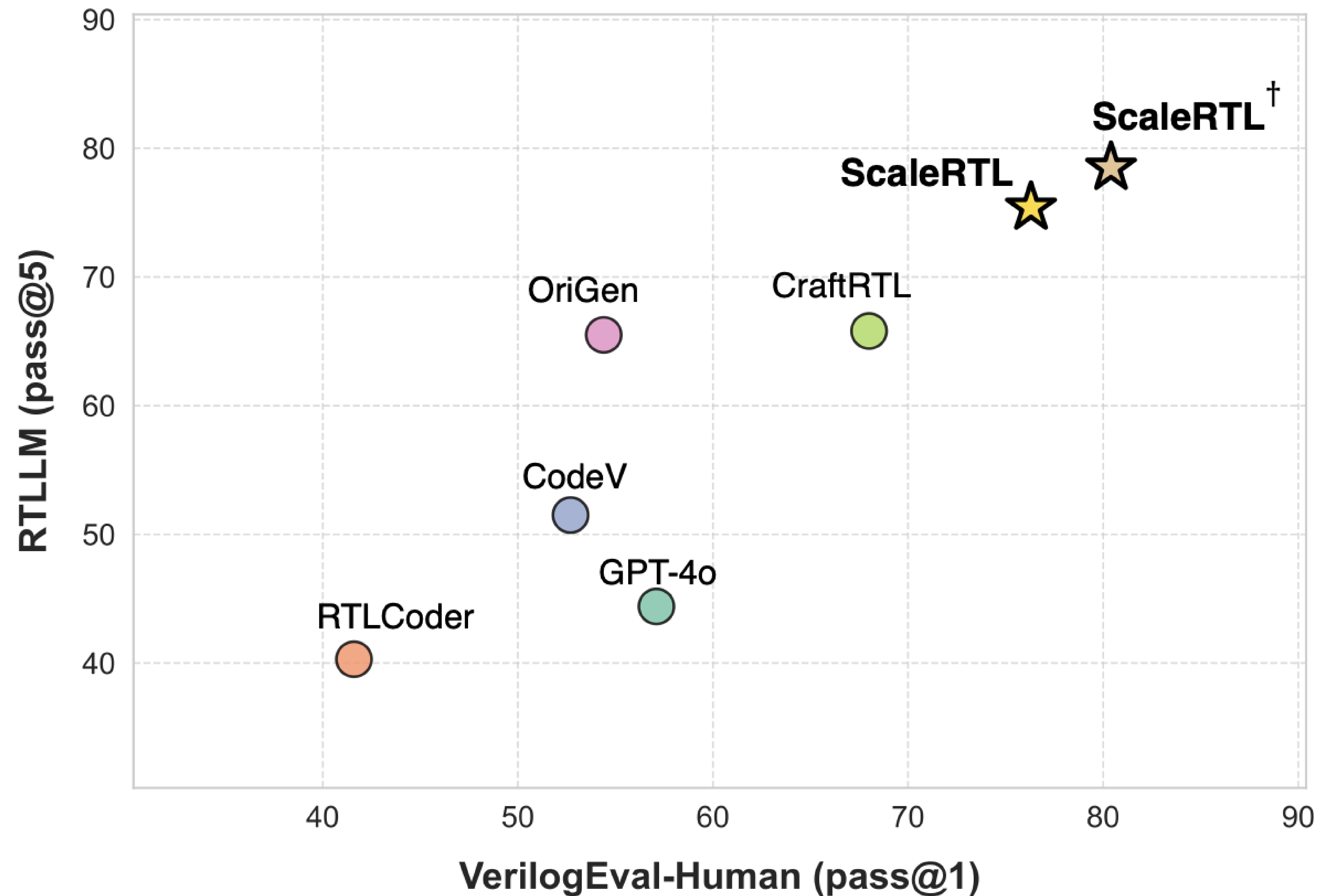


Iteratively extending
reasoning process
via corrective prompt

Model Output

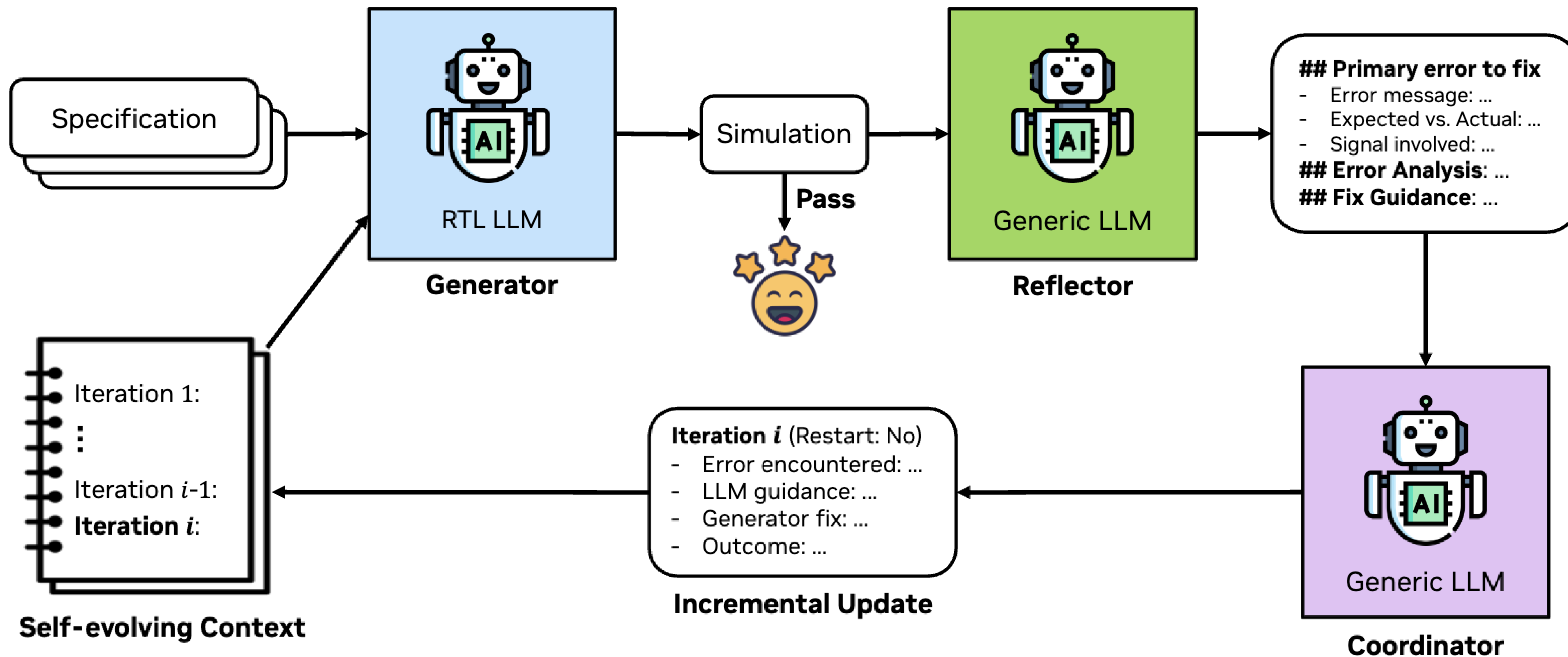
<think>
Alright, let's tackle this problem. The goal is ..., let's break it down step by step
* First, I need to model the seconds, minutes, and hours counters, ...
* Starting with the reset condition: when reset is high, the time should be ...
* Wait, but how to handle the BCD increment? Let's think ...
* But this could get complex. Alternatively, handle the entire increment as ...
* Wait, but how to handle BCD increment for hours. For example, ...
* ...
Yes. This covers the necessary transitions. So the code should work.
</think>
<answer>
[Code is omitted due to space constraints]
</answer>

ScaleRTL (32B) was the SOTA small LLM on VerilogEval



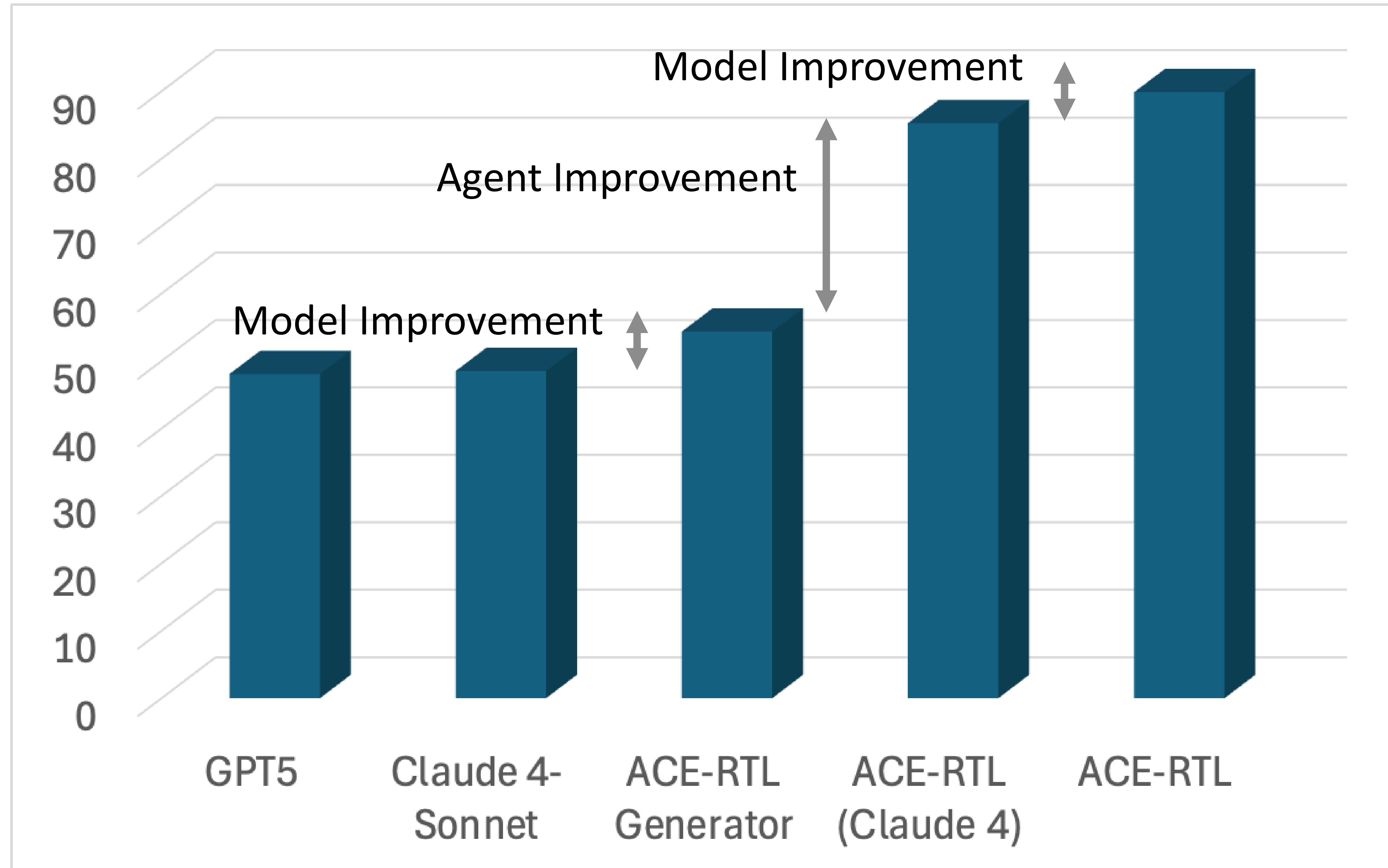
RTL Coding Tasks

Include a custom model in the agent



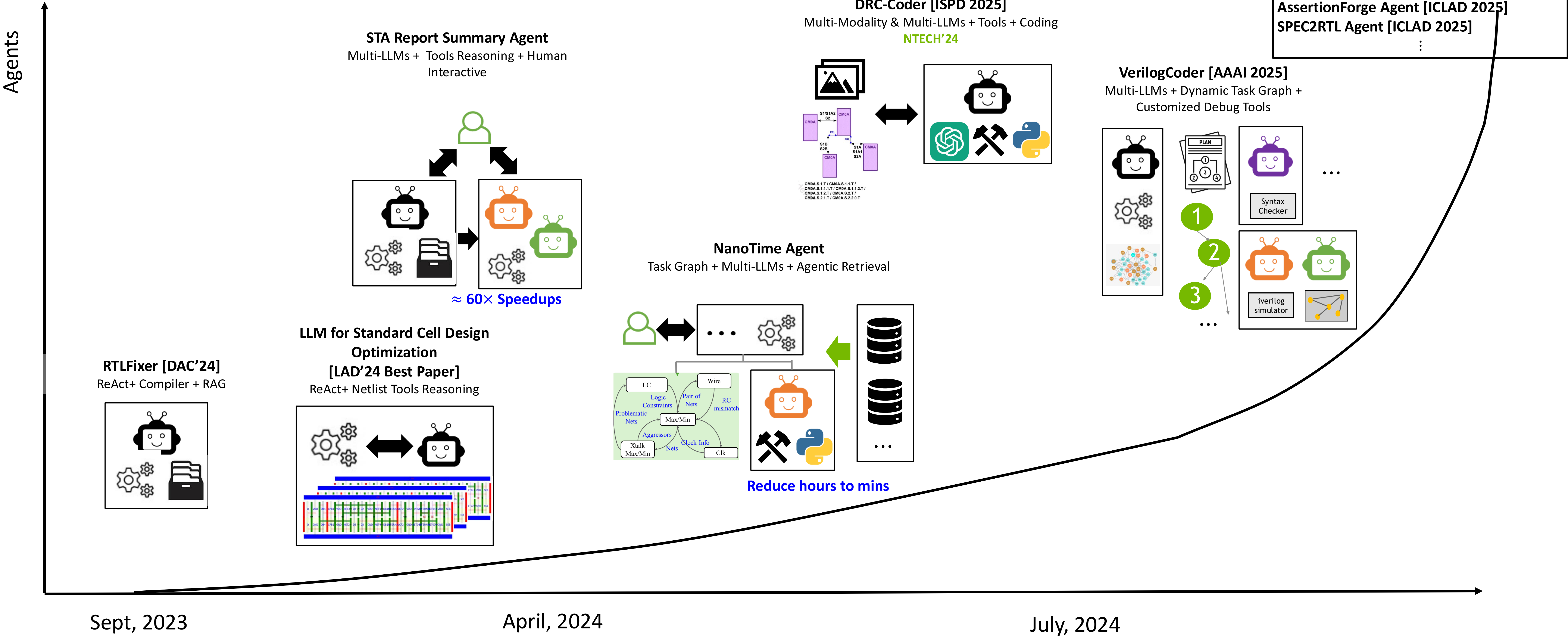
RTL Coding Tasks

Achieved 88.9% Pass Rate Across Four CVDP RTL Coding Tasks



Developing Custom Agents for Hardware Design @ NVIDIA

Lesson learned: Custom Agent Development Not Scalable

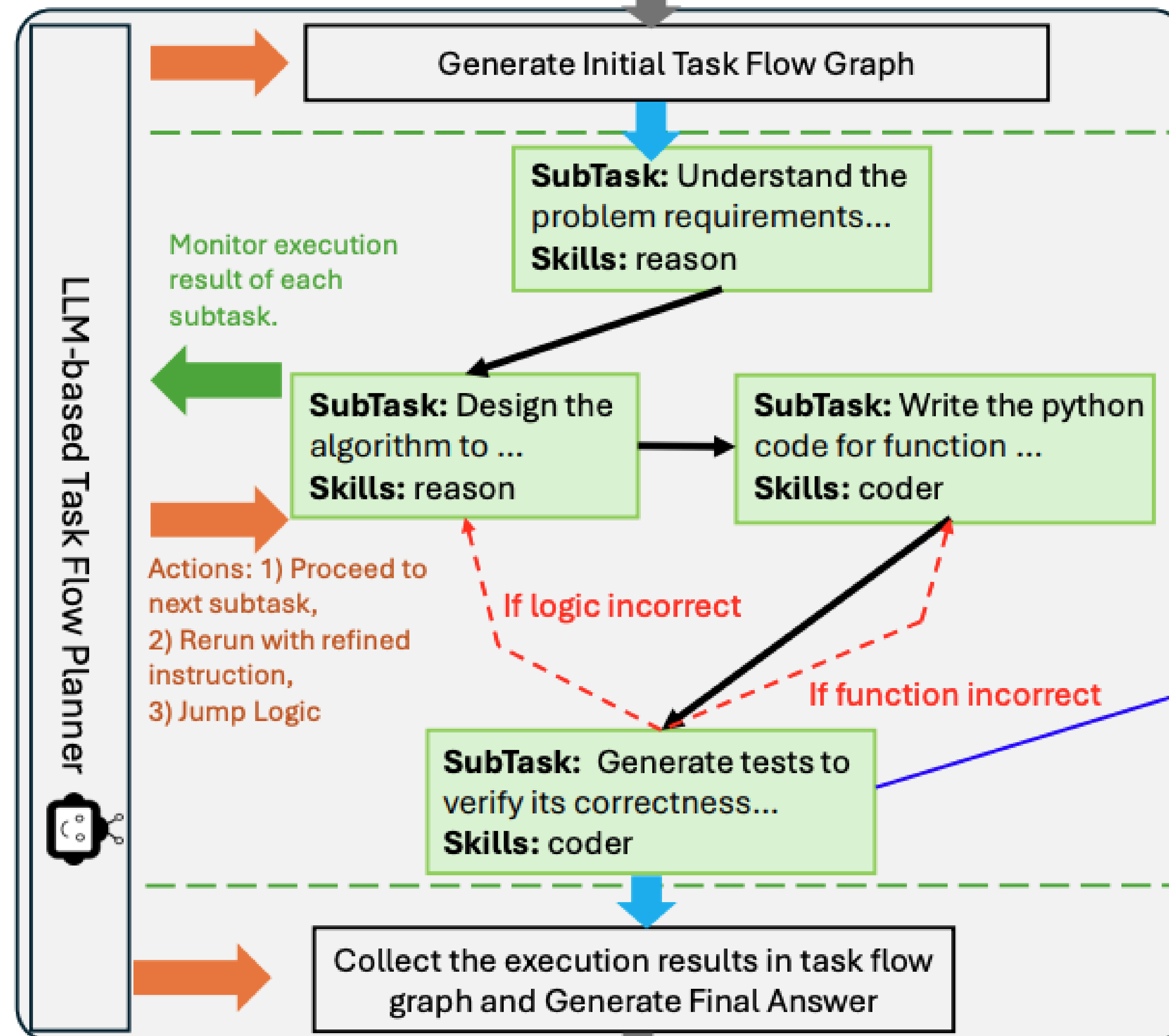


General Purpose Agent: Polymath Agent

Hierarchical Planning with graph and code plans

(a) An example of task flow graph solving

Task Prompt: You are a Python Coding expert. Complete the text_match_wordz function and return the completed code only...



Final Answer: def text_match_wordz(text): # Regular expression to match a word containing the letter ...

(b) An example of subtask code-represented workflow

SubTask: Generate tests to verify its correctness...

```
def workflow(instruction: str):
    instruction = "### Instruction Background ###\n" + instruction + "\n"
    <<<<<<< SEARCH
    ... # your designed workflow here.
    =====

    import json

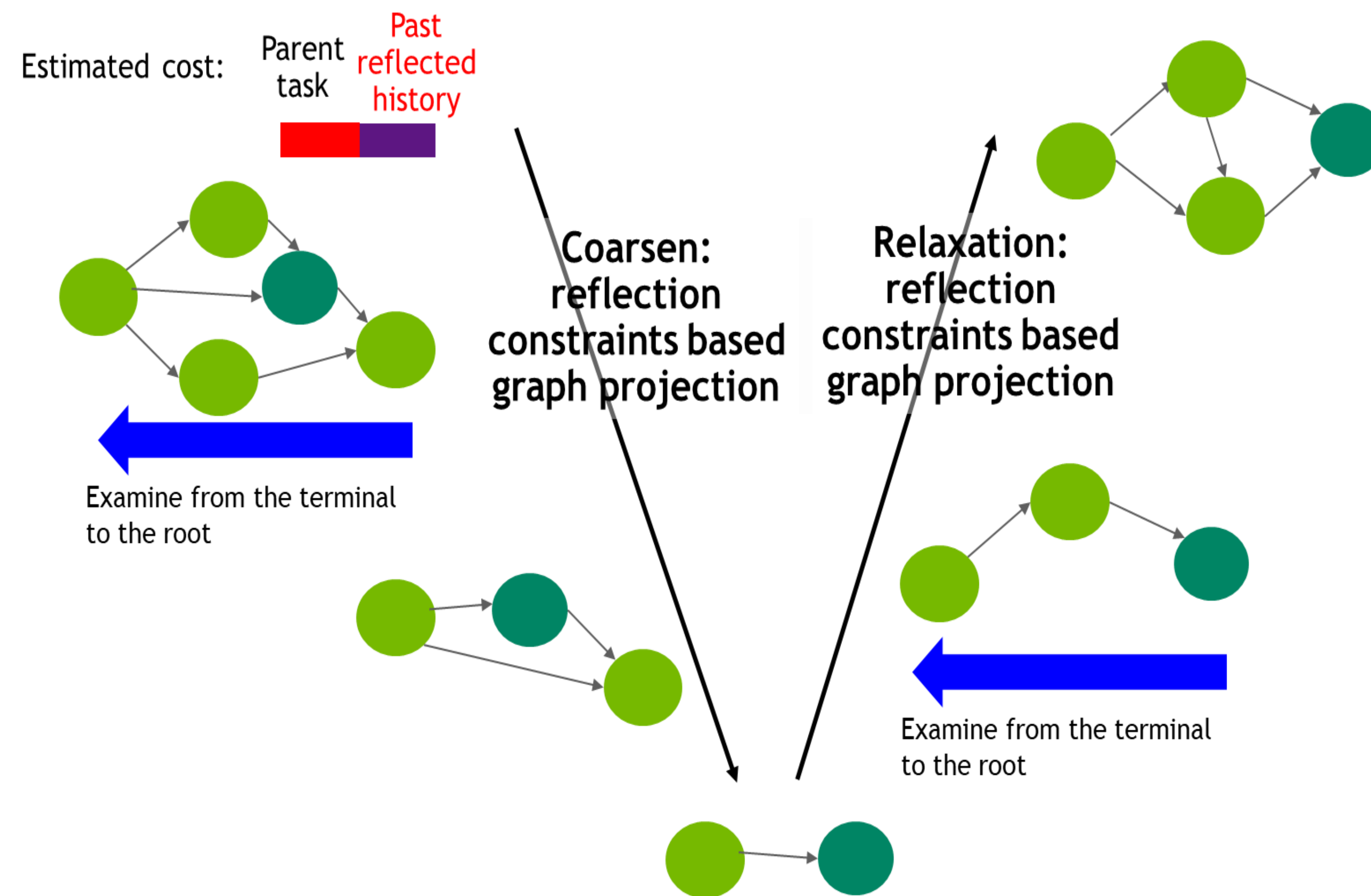
    # Task_3: Test the text_match_wordz function
    # Test inputs for the function
    test_cases = [
        "apple",
        "zebra",
        "buzz",
        "fizz",
        "hello"
    ]
    # Expected outputs: [False, True, True, True, False]
    expected_outputs = [False, True, True, True, False]

    # Chat with coder to run tests on function
    coder_instruction = f"Test the text_match_wordz function with inputs {test_cases}.\n"
    coder_instruction += "Ensure the function returns the correct boolean values indicating the presence of 'z' in words.\n"
    coder_instruction += "Return True for words containing 'z' and False otherwise.\n"
    response = coder_assistant.initiate_chat(message=instruction +
    coder_instruction)

    # Output the test results
    >>>>>>> REPLACE
    return response
```

Optimization of the Polymath Agent

Top-level Divide & Conquer: Manageable Subtask



Multi-grid based optimization based on LLM-as-judge

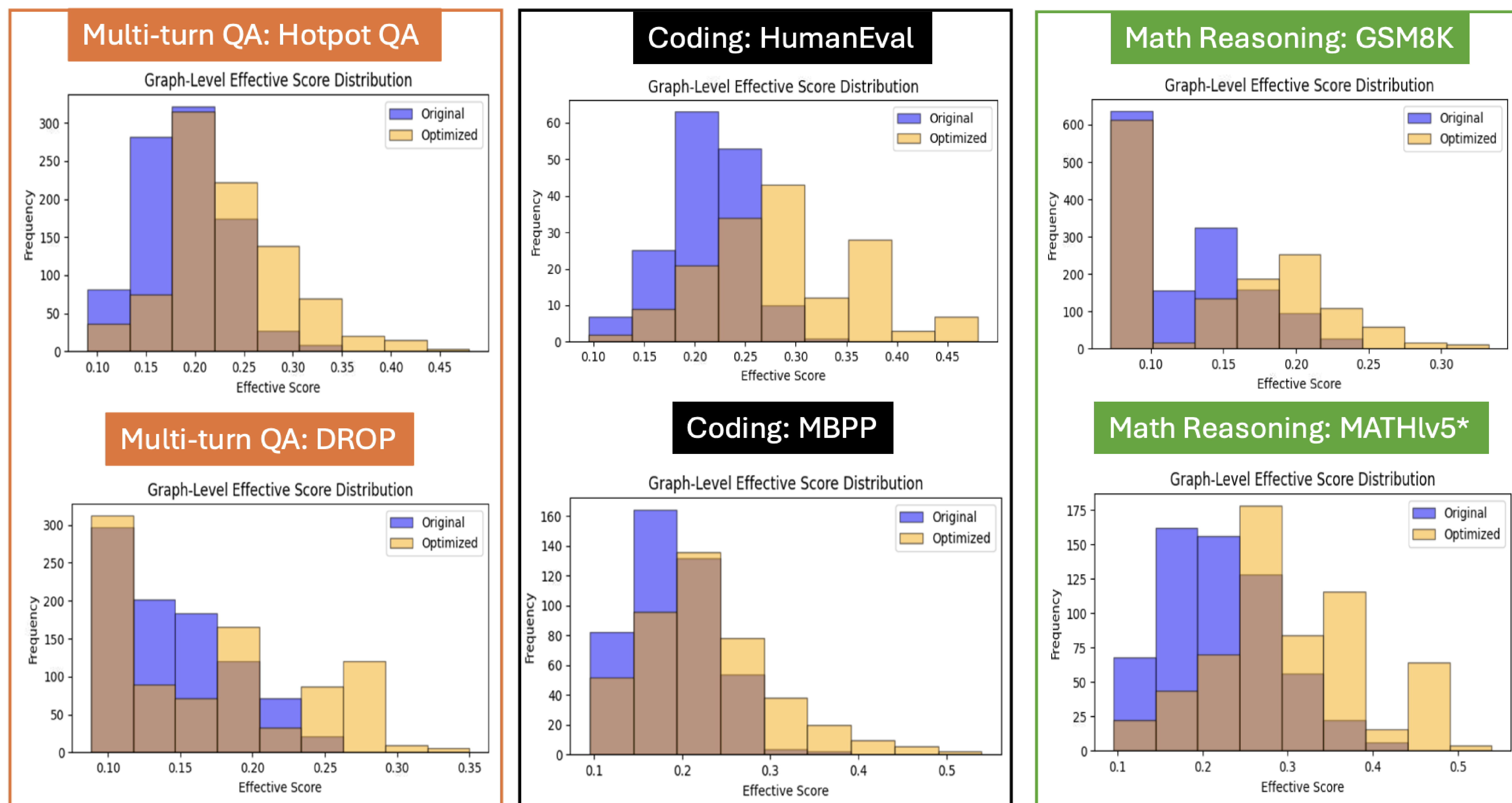
Subtask/Coding-level (Evolutionary Alg.) Optimize subtask execution & performance

```
39 # EVOLVE-BLOCK-START
40 def workflow(instruction: str):
41     """
42     Workflow to generate the 'drop_empty' function following the instruction input.
43     """
44     # Send the instruction directly to the coder assistant
45     coder_response = coder_assistant.initiate_chat(message=instruction)
46     coder_output = extract_output(
47         response=coder_response,
48         post_process_func=coder_description["post_process_func"]
49     )
50     # Return the coder's output directly
51     return coder_output
52 # EVOLVE-BLOCK-END
53
54 # Getting the strong reasoning
55 # Passing all reasoning to the coder assistant for testing implementation
56 strong_reason_debug_summary = strong_reason_debug_assistant.initiate_chat(message=full_task)
57 strong_reason_debug_summary = extract_output(
58     response=strong_reason_debug_summary,
59     post_process_func=reason_agent_description["post_process_func"]
60 )
61
62 # Sending the validated reasoning to coder assistant for testing implementation
63 coder_task = instruction + "\n" + "### Test Cases for 'drop_empty' ###" + strong_reason_debug_summary
64
65 coder_response = coder_assistant.initiate_chat(message=coder_task)
66 coder_output = extract_output(
67     response=coder_response,
68     post_process_func=coder_description["post_process_func"]
69 )
70
71 # Final string output as response
72 response = json.dumps({
73     "task": "Test the 'drop_empty' function with various dictionaries.",
74     "testing_results": coder_output
75 }, indent=4)
76
77 return response
78 # EVOLVE-BLOCK-END
```

Self-Reflection Guided Evolution

Optimization Improvement on Multiple Benchmarks

2%-8% performance improvements



Autonomous Multi-Agent Generation

Self-Improving Agent Solving RTL coding problems

Gen 0: Single-Agent

1 Agents + Tools + Skills

RTL Simulation & Debugging

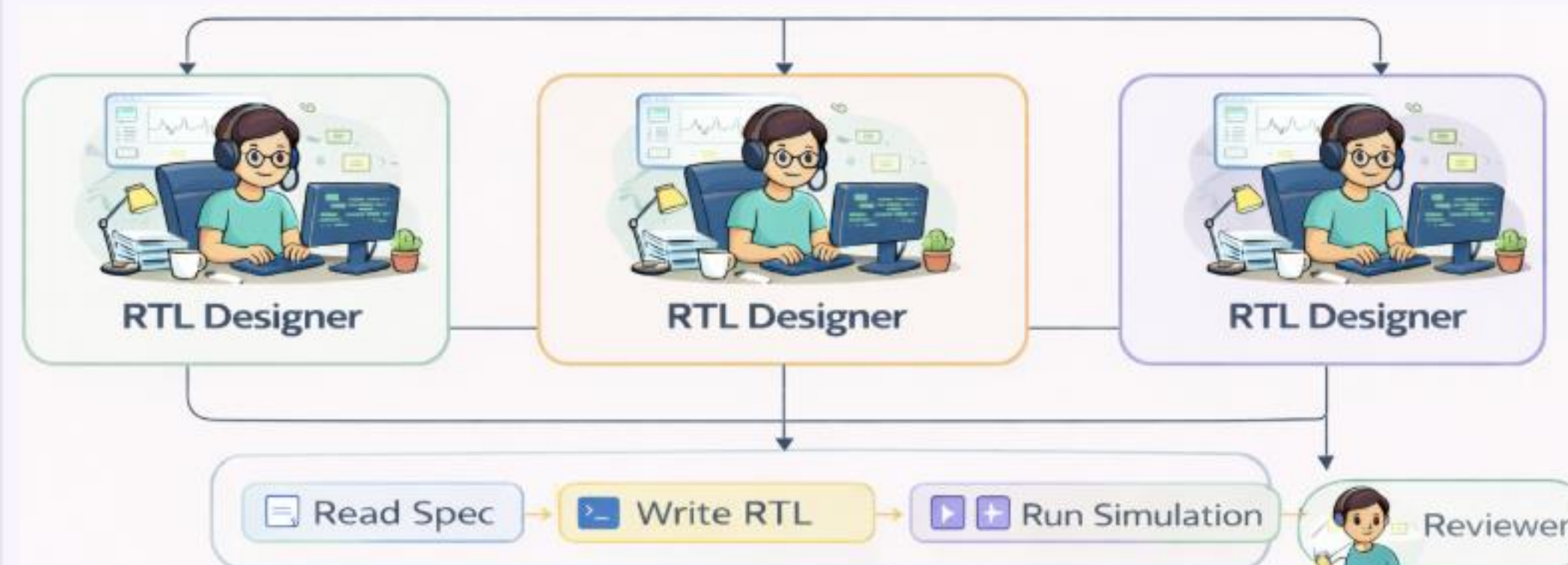


Read Spec → Write RTL → Simulate → Debug Loop

Gen 1: Multi-Agent System

5 Agents

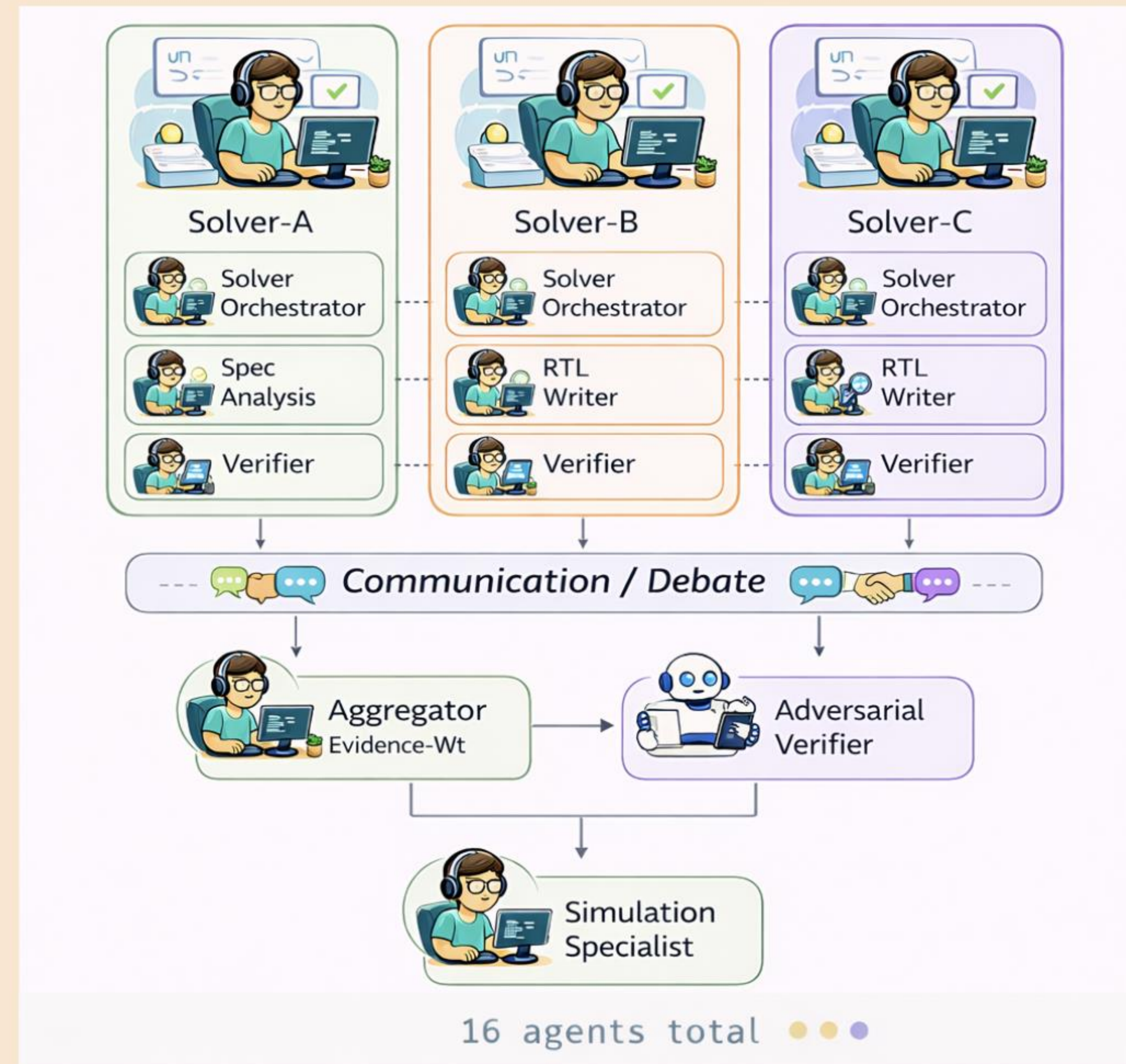
Orchestrator + 3 Designers + Reviewer



Gen 2: Hierarchical Agent Network

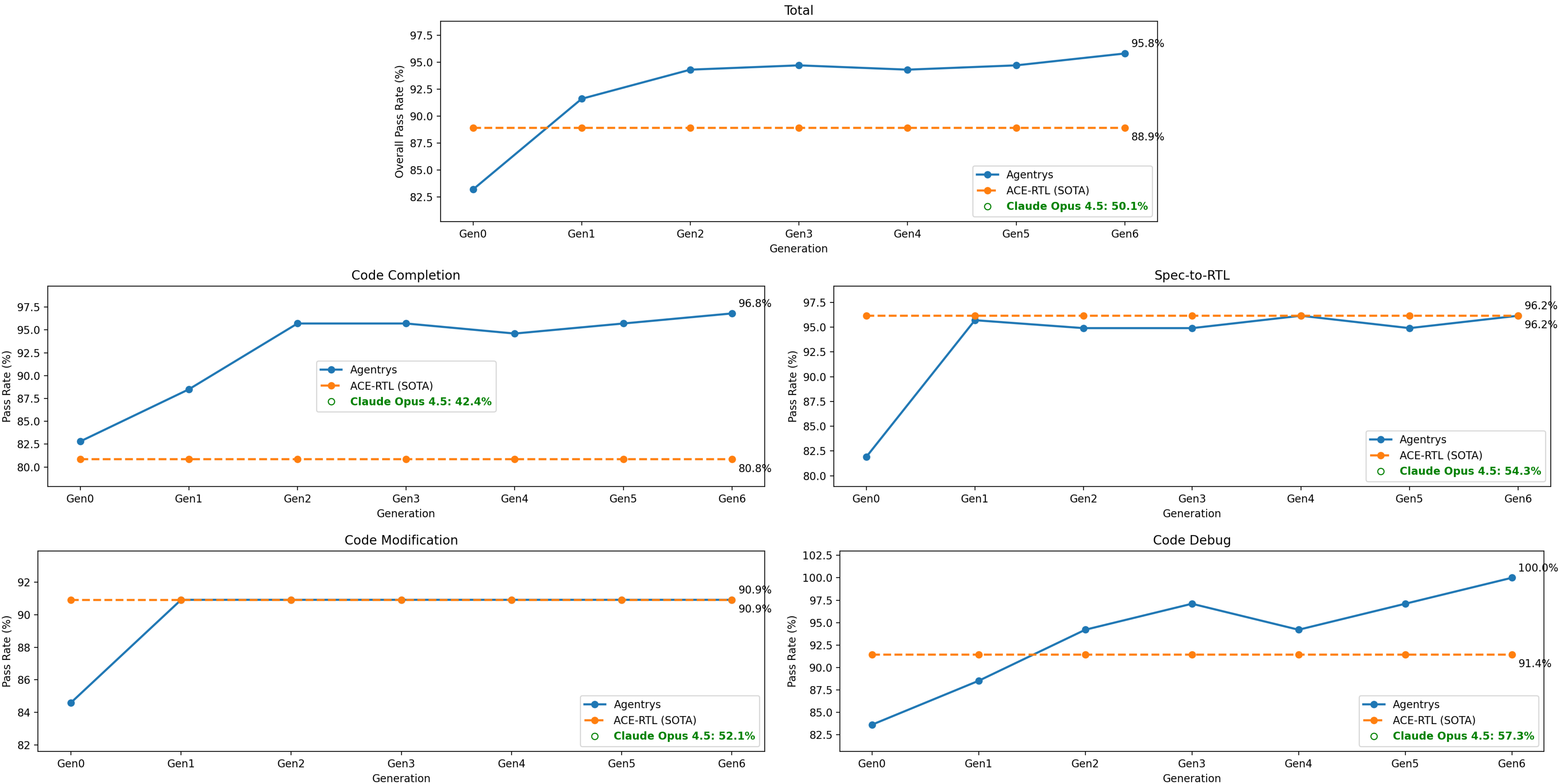
16 Agents

Orchestrator + 3 Solver Teams + Aggregator + Verifier + Specialist



Reaches 95.8% Overall Pass Rate with 100% on RTL Debug

7% better than hand-coded ACE-RTL Agent



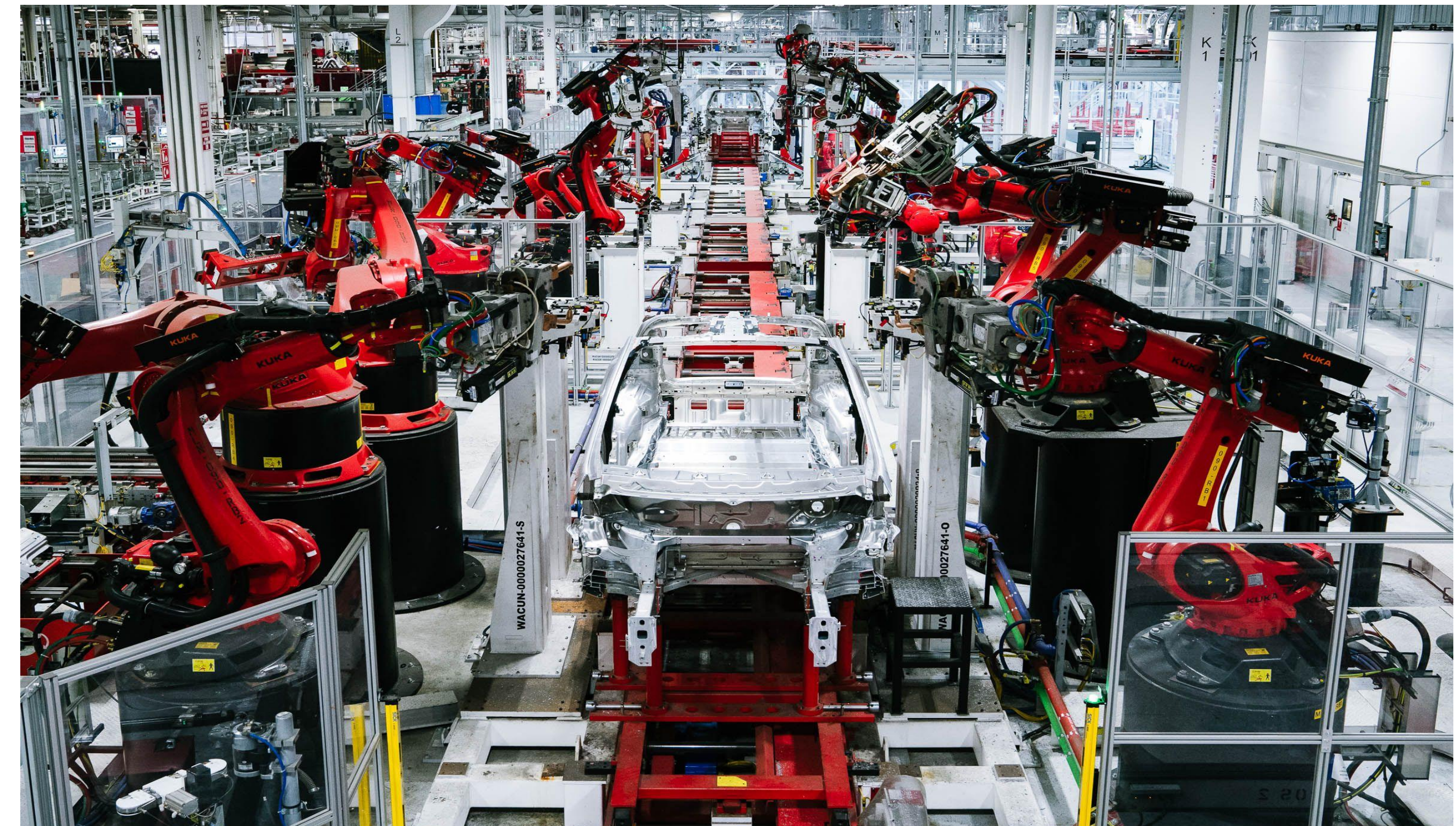
Toward Autonomous Design

Agentic Design Automation will lead us to autonomous design

Agent-native tools and custom models make agents more capable

Agents need to be optimized and improved to achieve the best performance

Future: Agents design chips. Designers develop the agents.





AGENTRYS enables design teams **develop** their own agentic design automation system and **learn** from their own data.

